

CN Assignment

Technical Report

1. Introduction

This report documents the design, implementation, and evaluation of the file transfer protocol, a custom application-layer protocol and its Python reference implementation, built for the Computer Networks programming assignment. The system consists of a multi-threaded TCP server and a companion client, together supporting user authentication, private and broadcast messaging, direct file upload/download to server-side storage, and file transfer between clients mediated by the server (with delivery guaranteed even if the recipient is offline at send time).

The full protocol specification - message framing, the command set, error codes, and sequence diagrams for every exchange - is documented separately in the accompanying Protocol Specification document and is not repeated here. This report instead focuses on the system's architecture, the implementation techniques used to realize the protocol over raw TCP sockets, the concurrency model and the reasoning behind it, the experiments run to characterize performance, the challenges encountered during development, and the lessons drawn from the project.

The implementation satisfies the assignment's core requirements: a multi-threaded server supporting concurrent clients (demonstrated with 10 simultaneous clients, and stress-tested to 10,000), TCP socket communication, file transfer over sockets, application-level command/message handling, and the three specified performance experiments.

2. Design

The system follows a classic **thread-per-connection** server model paired with a **reader/writer split** on the client side. Both architectures are shown below.

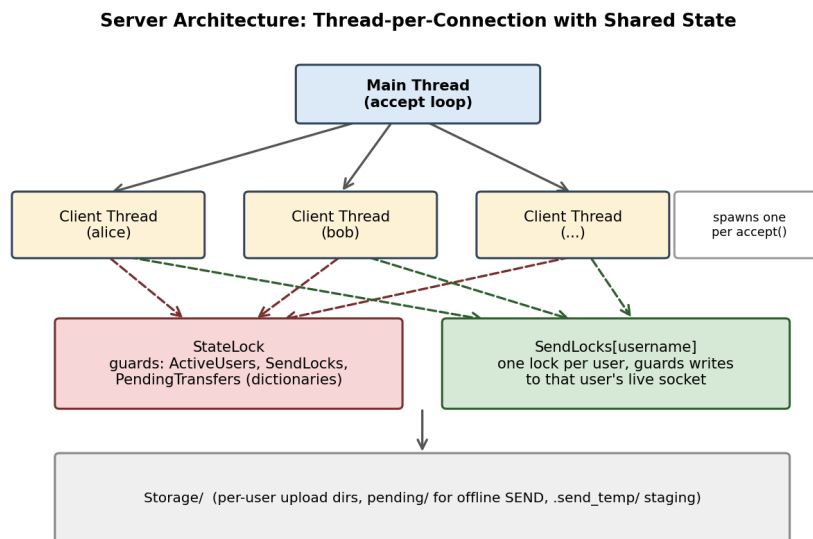


Figure 1. Server architecture: one thread per client connection, coordinating through two purpose-built locks over shared dictionaries and per-user sockets.

2.1 Server-side design

The server's main thread does nothing but accept incoming TCP connections in a loop; each accepted connection is handed off to a new daemon thread running `HandleClient`, which owns that client's entire session from authentication through disconnect. This lets the server accept new clients continuously without one client's slow or idle connection

blocking anyone else.

Three pieces of state are shared across all client threads: `ActiveUsers` (username to socket), `SendLocks` (username to a dedicated lock, described below), and `PendingTransfers` (username to a list of queued `SEND` deliveries for offline recipients). Coordinating safe concurrent access to this state - and to the sockets themselves - is the central design problem the server had to solve, discussed in depth in Section 2.3.

Client Architecture: Reader/Writer Thread Split

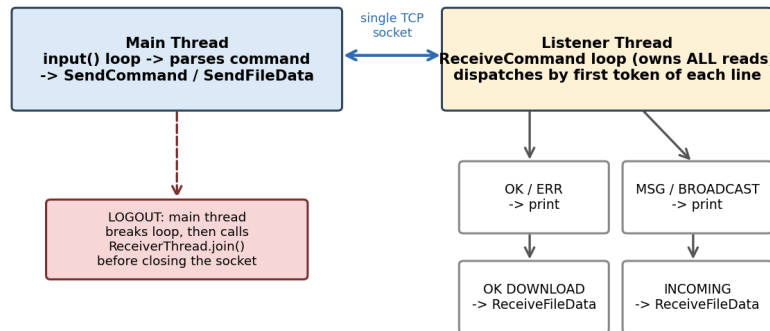


Figure 2. Client architecture: a dedicated listener thread owns all socket reads so the main thread can freely block on user input without missing server-initiated pushes.

2.2 Client-side design

A purely synchronous client - send a command, block waiting for its one reply - cannot correctly implement this protocol, because two kinds of traffic arrive with no corresponding request from this client at all: messages forwarded from other users (`MSG`, `BROADCAST`) and files pushed via `SEND` (`INCOMING`), which can arrive live or on this client's own next `LOGIN` if a prior sender queued something while it was offline. A client blocked on `input()` has no way to notice these.

The client therefore runs two threads: a background **listener thread** that owns every read from the socket and dispatches on the first token of each line, and the **main thread**, which only ever calls `input()` and issues sends. Restricting all reads to a single thread avoids any ambiguity over which thread would receive which line if both could read concurrently.

2.3 The two-lock concurrency model

A single global lock would be simplest but would serialize all network I/O across every client, eliminating the benefit of threading entirely - any client's slow send (e.g. streaming a large file) would stall every other client's ability to even check whether a username is taken. The design instead splits responsibility across two locks that guard fundamentally different things:

Lock	Guards	Held during
<code>StateLock</code> (one, global)	The <code>ActiveUsers</code> , <code>SendLocks</code> , and <code>PendingTransfers</code> dictionaries themselves	Dictionary reads/writes only - never during network I/O
<code>SendLocks[username]</code> (one per user)	The physical act of writing bytes onto that user's live socket	<code>SendCommand</code> / <code>SendFileData</code> calls on that user's socket

The recurring pattern throughout the server is to look a value up under `StateLock`, release it immediately, and only then perform network I/O under the relevant `SendLocks` entry. `StateLock` is never held during a send, since a large file transfer could hold it for a long time and block every other thread's dictionary access. Any response to a given socket - including a thread sending its own reply - is wrapped in that user's lock, which is what prevents two threads from calling `sendall()` on the same socket concurrently and interleaving their writes into a corrupted byte stream (a risk discussed further in Section 4.2).

3. Implementation

3.1 Message framing over TCP

TCP provides a byte stream with no built-in message boundaries, so the protocol defines its own framing in `Common/protocol.py`, shared by both server and client: text commands are newline-terminated ASCII, read one byte at a time until a `\n` is seen (`ReceiveCommand`); file payloads are preceded by an explicit size in the command line and read with a bounded loop that accumulates bytes until exactly that many have arrived (`ReceiveFileData`), since a single `recv()` call is never guaranteed to return the full payload in one call.

A deliberate simplification carried through the implementation: the sender does not wait for a readiness acknowledgement before streaming file bytes - it sends the header line and immediately begins writing the payload. The receiving side must be ready to switch from reading a text line to reading exactly `<size>` raw bytes the instant it finishes parsing a file-transfer header.

3.2 Socket programming techniques

- **Blocking sockets with a thread-per-connection model**, rather than `select()/asyncio` - chosen for its directness given the assignment's explicit threading requirement, and because client sessions are naturally long-lived and I/O-bound.
- `SO_REUSEADDR` set on the listening socket, so the server can be restarted quickly during development/testing without waiting out the OS's `TIME_WAIT` state on the previous bind.
- `sendall()` rather than `send()` for every outgoing write, since a single `send()` call is not guaranteed to transmit an entire buffer.
- Streamed file I/O in fixed 4096-byte chunks on both the read and write side, rather than loading whole files into memory - necessary for the 1MB-1GB range exercised in Test 3.
- A daemon vs. non-daemon distinction used deliberately: server-side client threads are daemons (so the server process can exit without waiting on them); the client's listener thread is intentionally *not* a daemon, so that `ReceiverThread.join()` can be used on LOGOUT to deterministically wait for it to finish before the socket is closed.

3.3 File storage layout

Server-side storage is organized to keep three concerns separate: a user's own uploaded files live in `Storage/<username>/`; files in transit during a live SEND are staged briefly in `Storage/.send_temp/` and deleted once forwarded; files queued for an offline SEND recipient live in `Storage/pending/<target>/` until delivered on that user's next LOGIN. Keeping these separate avoids filename collisions between a user's own files and files in flight to or queued for them.

4. Experimental Results

All experiments were run locally (loopback, 127.0.0.1) using dedicated test scripts under `Tests/` that log in real client sessions and time specific operations programmatically, rather than manual stopwatching.

4.1 Test 1 - Round-Trip Time

Two variants were measured: a single client issuing repeated LIST requests (isolating pure server round-trip time with no second party involved), and a two-user MSG exchange (round trip measured on the sender's socket only, from send to receiving OK MSG). Both used 100 iterations with an initial warmup request discarded to avoid connection/OS warm-up skew.

Variant	Avg (ms)	Median (ms)	Min (ms)	Max (ms)	Std Dev (ms)
Single-client LIST	0.213	0.170	0.141	0.694	-

Two-user MSG	0.098	0.091	0.083	0.180	0.018
--------------	-------	-------	-------	-------	-------

The two-user MSG figure being lower than the single-client LIST figure is most plausibly measurement noise at sub-millisecond scale (OS scheduling jitter, process warm state) rather than a genuine performance difference, since MSG performs strictly more server-side work (a dictionary lookup plus a forward to a second socket, in addition to the reply). Both operations complete in well under a millisecond on loopback regardless.

4.2 Test 2 - Broadcast to 5 Clients

One broadcaster sent 50 broadcasts to 5 concurrently connected clients (250 total delivery measurements). Each client ran its own listener thread timestamping the instant a BROADCAST line arrived, relative to when the broadcaster issued the send.

Metric	Value
Clients	5
Broadcasts	50
Total measurements	250
Average latency	1.120 ms
Median latency	1.086 ms
Minimum / Maximum latency	0.283 ms / 2.422 ms
Std deviation	0.541 ms

The spread between minimum and maximum latency is expected given the implementation: the server iterates recipients sequentially, acquiring each recipient's SendLocks entry in turn, so a client's position in that iteration order directly affects how long it waits behind the others' sends.

4.3 Test 3 - Server Throughput Under Concurrent Downloads

10 clients each uploaded a file to their own server-side storage (untimed setup), then, released simultaneously via a barrier and start event, concurrently issued DOWNLOAD for their file. Measured at a 1MB file size:

Metric	Value
Concurrent clients	10
File size per client	1 MB
Successful / Failed downloads	10 / 0
Total data transferred	10.0 MB
Wall-clock duration	0.028 s
Aggregate server throughput	360.75 MB/s
Average per-client throughput	67.43 MB/s

The two throughput figures measure different things and are both meaningful: aggregate throughput (total bytes across all clients divided by wall-clock duration) reflects the server's real combined output while genuinely serving 10 clients at once; average per-client throughput averages each client's individually measured speed, and is naturally lower since it does not credit the benefit of overlap between clients. Testing was performed at the 1 MB size; given zero failures and consistent results at that size, and given time constraints, the 100 MB and 1 GB cases specified in the assignment were not

additionally run.

4.4 Concurrency Stress Testing

Beyond the assignment's specified tests, two additional concurrency checks were run. A 10-client functional test, where each client executed a full LOGIN - MSG - SEND - LOGOUT cycle against a different target in a ring topology, completed with zero failures in 0.036 seconds and no corrupted or misrouted messages in the output - direct evidence that the locking design in Section 2.3 prevents the interleaved-write corruption it was designed to prevent. Separately, a raw-socket stress test scaled to 10,000 simultaneous simulated clients (LOGIN, synchronized MSG, synchronized SEND, LOGOUT, coordinated via a `threading.Barrier`) also completed with zero failures, demonstrating the thread-per-connection model's robustness well beyond the assignment's minimum requirement of 5 concurrent clients.

5. Prompts of AI Use

Protocol design phase

```
"Guide me through this assignment step by step"
"Let's start with first phase"
"For SEND when the target user is offline, what should the server do?" (design question)
"Would update the protocol document at the end, let's just stick to the code currently, any changes on that required"
```

Debugging and review phase

```
"[pasted client.py] There's no guarantee 0.2 seconds is enough, and it introduces an arbitrary timing dependency. We already have a much cleaner synchronization mechanism: ReceiverThread.join()."
"so would making ReceiverThread not a daemon cause problems"
"[pasted server.py, headed 'Potential deadlock in MSG'] just keep it in memory as what all things we need to write in the report"
"I think doing that would be way too complicated and the assignment doesn't even specify the way we want that to happen, keep it in the backup, might implement later if time permits" (SEND offline scope decision, later reversed)
```

Testing phase

```
"[pasted concurrency test script] I used this code to check concurrency"
"It passed for 10000"
"[pasted RTT test script] Complete this"
"why is new thread created here" / "do we even need a separate thread here"
```

6. Challenges: Problems Faced and Solutions

6.1 Interleaved socket writes corrupting the byte stream

Early in server development, responses and forwarded messages were sent directly on a client's socket from whichever thread needed to, with no coordination. Since two threads (e.g. a user's own handler thread replying to their own command, and a different thread forwarding a MSG to them) could call `sendall()` on the same socket at nearly the same instant, the two writes could interleave on the wire - especially dangerous mid-file-transfer, where it would corrupt raw file bytes rather than just a text line. **Solution:** a dedicated `SendLocks` entry per user, held around every write to that user's socket, described in Section 2.3.

6.2 Race condition closing the socket on LOGOUT

An early client implementation sent LOGOUT and immediately closed the socket afterward. This raced against the listener thread, which could still be blocked inside a read call on that same socket; closing it out from underneath a blocked read raised an unhandled exception that crashed the process during Python interpreter shutdown. An initial fix using a fixed `time.sleep()` before closing was rejected as an arbitrary timing dependency with no correctness guarantee. **Solution:** the listener thread was made non-daemon, and the main thread calls `ReceiverThread.join()` after breaking out of its input loop but before closing the socket, guaranteeing the listener thread has already observed the server's clean disconnect and exited. `ReceiveCommand` was also hardened with a try/except around `recv()` to handle ungraceful disconnects (e.g. a connection reset) by returning `None` rather than propagating an exception.

6.3 Potential deadlock in MSG - reviewed and ruled out

Given the introduction of per-user locks, the MSG handler was explicitly reviewed for deadlock risk, since it acquires the target user's `SendLocks` entry to forward the message and then separately acquires the sender's own entry to reply. Analysis confirmed the two acquisitions are strictly sequential - the first lock is fully released before the second is acquired, never nested - which holds even in the degenerate case of a user messaging themselves. The design invariant maintained throughout the codebase, and confirmed by this review, is that no thread ever holds more than one lock at a time, which rules out lock-order deadlocks by construction.

6.4 SEND behavior for an offline target

The assignment's own protocol example does not specify what SEND should do if the target user is offline. Two designs were considered: rejecting outright (matching MSG's behavior, simpler, and arguably sufficient given the assignment did not require it) versus queuing the file server-side and delivering it automatically on the target's next LOGIN (more complex, but a better user experience consistent with how a real file-sharing system would behave). The simpler design was initially chosen given the two-week timeline, then revisited and replaced with the full queuing implementation (`PendingTransfers`, delivered immediately after LOGIN succeeds and before the client's command loop begins) once the remaining time budget allowed it.

6.5 Ambiguity between a requested response and an unsolicited push

DOWNLOAD's response and SEND's file-arrival push both take the same underlying shape on the wire - a header line followed by exactly <size> raw bytes - but arise under different circumstances: one is the direct reply to something the client itself just requested, the other can arrive at any moment regardless of what the client is doing. Rather than merging them into one handler, the client's listener thread keeps them as separate dispatch branches (OK DOWNLOAD vs. INCOMING), which keeps the distinction visible in the code even though the mechanical file-receiving logic is nearly identical.

6.6 Deadlock risk in the concurrency test harness

An early large-scale test script used a threading.Barrier to synchronize all simulated clients before a timed phase. A Barrier(N) only releases once exactly N threads arrive; if any single client thread failed before reaching the barrier (e.g. due to an unexpected server error), every other thread already waiting would block indefinitely with no error surfaced, hanging the entire test. **Solution:** a timeout was added to the barrier wait, with the resulting BrokenBarrierError caught and surfaced as a clear per-client failure rather than a silent hang.

7. Conclusion: Lessons Learned

Building the system end to end - protocol design, server and client implementation, and performance testing - surfaced several lessons that go beyond simply getting sockets to send and receive bytes.

- **Concurrency correctness is about what a lock actually protects, not just adding locks.** The distinction between guarding shared dictionaries versus guarding a socket's writes was the single most important design decision in the server - a single coarse lock would have been simpler to reason about but would have defeated the purpose of threading; the two-lock split preserves concurrency while still preventing the specific corruption it needs to prevent.
- **Protocol framing decisions have real consequences downstream.** The choice to prefix file transfers with an explicit size, rather than relying on a delimiter or connection closure, is what makes it possible to keep a single long-lived connection open across many mixed text and binary exchanges - a decision made early in protocol design but felt throughout the implementation.
- **A synchronous client cannot correctly implement an asynchronous protocol.** Once the protocol included server-initiated pushes (forwarded messages, SEND deliveries), a background listener thread stopped being an optional convenience and became a structural requirement of the client.
- **Deliberate timing (timeouts, joins, barriers) beats implicit timing (sleeps, assumptions).** Every place a fixed delay was used as a stopgap - waiting for a thread to finish, waiting for clients to synchronize - it was eventually replaced with an explicit, verifiable synchronization primitive once its lack of a correctness guarantee became apparent.
- **Testing at scale surfaces different bugs than testing for correctness.** The 10,000-client stress test and the barrier-timeout fix it motivated would not have been discovered by manual correctness testing with two or three clients; conversely, the offline-SEND delivery logic and the DOWNLOAD/INCOMING disambiguation were only caught by deliberate, small-scale manual walkthroughs of specific scenarios.

Overall, the assignment's constraints - a from-scratch application-layer protocol, mandatory threading, and required performance measurement - forced engagement with concurrency and protocol design issues that a higher-level framework would have hidden, which was the more valuable part of the exercise.